

Persistency (Save/Load)

Of course if anything is created in a game that should be able to be saved. The Advanced Building System does not provide any persistence functionality, just the possibility to save(serialize) and load(deserialize) everything that the player has been built.

Default.....	2
Serialization/Deserialization.....	2
Serialization of BuildingParent.....	3
Deserialization of the BuildingParents.....	3
Serialization of Building.....	4
Deserialization of the Buildings.....	4
Serialization of BuildingElement.....	6
Deserialization of BuildingElement.....	6
ABS_PersistencyLoadErrorCode.....	7

Default

During the builtin Start function of the MonoBehaviour, the Buildings and the BuildingParents are collecting their childs and sign them as default. This feature is implemented for avoiding the duplication of those objects that are already placed into the scene and not created during the runtime of the game. If those elements would be recreated during the load process they would be duplicated because those objects are already presented at the start of the scene. The persistence feature saves that information which objects are default during the serialization process. During the load process the default element list is double checked.

- If an element is default in the scene but not included by the serialized data that element will be destroyed. The same applied for the default buildings.
- If an element or a building is default and included by the serialized data it will be updated not re-created.

Serialization/Deserialization

Every Building is inherited from the SaveableMonobehaviour which has this abstract function called `GetPersistedData`. This function gives back a `PersistedDataClass` that can be persisted by the developer or you can use the `PersistencyManager` to persist the Building into JSON string which can be saved easily.

The “Save To File” button on the Building’s interface is using the same method.

```
string dataToSave = PersistencyManager.ToJson(target.GetPersistedData());
```

The loading is the same just backward. The `PersistencyManager` can be used to create the `PersistedData` from the JSON text and use the Building’s `LoadFromPersistedData` function to create the Building.

Or just use the `CreateFromJSON` function which is a static function of the Building class which can create a Building from a json text. Just like the `BuildingLoader` tool.

Serialization of BuildingParent

The `ABS_BuildingParent` is inherited from the `ABS_SaveableMonobehaviour` so it implements the `ToJSON` function which can be used to get the serialized data of the building parents with all of its building and their elements as well.

It is possible to get this data object with the `GetPersistedData` function. It is a public function but it is not recommended to use it.

Functions for serialization:

- `ToJSON`
- `GetPersistedData`

`ABS_BuildingParentPersistedData`'s persisted data example:

```
{
  "m_Type": 3,
  "m_Name": "Buildings",
  "m_InstanceGuid": "cc723beb-5df6-4f3d-9948-d25580a27255",
  "m_PrefabGuid": "5c3d9c87-797d-489b-8a59-4b5d7e259ac9",
  "RemainingDefaultBuildings": [
    "826a4671-06bd-40e2-8cfe-1f8f749864ee",
    "9227f5cc-473c-42d2-b441-943e2cf3ecda",
    "af095aa7-e0ef-4cb0-bcc6-915602e76bef"
  ],
  "GlobalBasicGridParentGuid": "9227f5cc-473c-42d2-b441-943e2cf3ecda",
  "GlobalFreeParentGuid": "826a4671-06bd-40e2-8cfe-1f8f749864ee",
  "FreeBuildings": [ ... ],
  "BasicGridBuildings": [ ... ],
  "AdvancedGridBuildings": [ ... ],
  "SnapPointBasedBuildings": []
}
```

Deserialization of the BuildingParents

For the deserialization process so for the loading of the saved building the following options are available:

- `UpdateFromPersistedJSON` (static)
- `UpdateFromPersistedData` (static)
- `CreateFromJSON` (static)
- `CreateFromPersistedData` (static)
- `CreateFromPersistedData`
- `UpdateFromPersistedData`

Serialization of Building

The ABS_Building is inherited from the ABS_SaveableMonobehaviour so it implements the ToJSON function which can be used to get the serialized data of the building with all of its elements. During the serialization process a persisted data object will be created from the following list:

- ABS_FreeBuildingPersistedData
- ABS_BasicGridBuildingPersistedData
- ABS_AdvancedGridBuildingPersistedData
- ABS_SnapPointBasedBuildingPersistedData

All of them are inherited from the ABS_BuildingPersistedData which contains the building's basic properties. These data objects will be serialized into a json string.

It is possible to get this data object with the GetPersistedData function. It is a public function but it is not recommended to use it. It is meant to be used during the persistence process. It is used for the serialization of the BuildingParent.

Usually the buildings are saved with the building parents too but with the ToJSON function there is the possibility to save only one building's data.

Functions for serialization:

- ToJSON
- GetPersistedData

ABS_AdvancedGridBuildingPersistedData's persisted data example:

```
1      "AdvancedGridBuildings": [  
2          {  
3              "m_Type": 2,  
4              "m_Name": "Building1",  
5              "m_InstanceGuid": "af095aa7-e0ef-4cb0-bcc6-915602e76bef",  
6              "m_PrefabGuid": "560f611f-2aa2-492a-9978-02c1caa137af",  
7              "PositionSearchAlgorithmType": 2,  
8              "WorldPosition": {  
9                  "x": -15.835441589355469,  
10                 "y": 0.5,  
11                 "z": 5.501210689544678  
12             },  
13             "WorldEulerAngles": {  
14                 "x": 0.0,  
15                 "y": 342.5999755859375,  
16                 "z": 0.0  
17             },  
18             "MaximumElementCount": 1000,  
19             "EnableCache": false,  
20             "UpperRangeLimitEnabled": false,  
21             "UnderRangeLimitEnabled": false,  
22             "SideRangeLimitEnabled": false,  
23             "UpperRangeLimit": 1000.0,  
24             "UnderRangeLimit": -1000.0,  
25             "SideRangeLimit": 1000.0,  
26 >             "RemainingDefaultElements": [...  
47         ],  
48 >             "Elements": [...  
389         ],  
390         "BuildingPositionModifier": {  
391             "x": 0.0,  
392             "y": 0.0,  
393             "z": 0.0  
394         }  
395     },
```

Deserialization of the Buildings

For the deserialization process so for the loading of the saved building the following options are available:

- **CreateFromPersistedJSON (static)**
- **CreateFromPersistedData (static)**
- **CreateFromPersistedData**
- **UpdateFromPersistedData**
- **CheckUpdatePossibility**

The **CreateFromPersistedJSON (static)** is a static function that will first deserialize the given json string and create a Building based on the deserialized PersistedData object. This element will be automatically added to the target BuildingParent. The newly created building will be returned within the p_NewBuilding out argument of the function.

The **CreateFromPersistedData (static)** is a static function that will create a building based on the given PersistedData object. This building will be automatically added to the target BuildingParent. The newly created element will be returned within the p_NewBuilding out argument of the function.

The **UpdateDataFromPersistedData** function is used for updating the state of a building element. It is used for updating the default elements that are already in the scene and not created during the load process.

The **CreateFromPersistedData** function is working like the update function. The only difference is that in this function the algorithm starting from that position the building is just created so everything should be created. Meanwhile the update will check what should be updated and what should be created. Something like a force update.

The **CheckUpdatePossibility** function is used for a precheck of the PersistedData. If the check fails the data object can not be used to update the element.

Serialization of BuildingElement

The `ABS_BuildingElement` is inherited from the `ABS_SaveableMonobehaviour` so it implements the `ToJSON` function which can be used to get the serialized data only for the element. During the serialization process an `ABS_BuildingElementPersistedData` is created that contains the data of the building element. This data object will be serialized into a json string.

It is possible to get this data object with the `GetPersistedData` function. It is a public function but it is not recommended to use it. It is meant to be used during the persistence process. It is used for the serialization of the buildings.

Usually the building elements are saved with the buildings too but with the `ToJSON` function there is the possibility to save only one building element's data.

Functions for serialization:

- `ToJSON`
- `GetPersistedData`

`ABS_BuildingElement`'s persisted data example:

```
1  {
2      "LocalPosition": {
3          "x": 0.0,
4          "y": 0.0,
5          "z": 0.0
6      },
7      "LocalEulerAngles": {
8          "x": 0.0,
9          "y": 0.0,
10         "z": 0.0
11     },
12     "Data": {
13         "m_Type": 1,
14         "m_Name": "StoneFoundation(Clone)",
15         "m_InstanceGuid": "2405c798-889d-457d-babd-fb9353c3c4c1",
16         "m_PrefabGuid": "823a2c25-a48b-4df6-ad38-d387980c6b2b",
17         "Properties": 2
18     }
19 }
20
```

Deserialization of BuildingElement

For the deserialization process so for the loading of the saved `BuildingElement` the following options are available:

The **`CreateFromPersistedJSON`** is a static function that will first deserialize the given json string and create a `BuildingElement` based on the deserialized `PersistedData` object. This element will be automatically added to the target `Building`. The newly created element will be returned within the `BuildingElement` out argument of the function.

The **`CreateFromPersistedData`** is a static function that will create a `BuildingElement` based on the given `PersistedData` object. This element will be automatically added to the target `Building`. The newly created element will be returned within the `buildingElement` out argument of the function.

The **UpdateDataFromPersistedData** function is used for updating the state of a building element. It is used for updating the default elements that are already in the scene and not created during the load process.

The **CheckUpdatePossibility** function is used for a precheck of the PersistedData. If the check fails the data object can not be used to update the element.

ABS_PersistencyLoadErrorCode

Every deserialization related function returns an error code.

The goal of the error code is to show the reason behind that if the deserialization process fails.

```
public enum ABS_PersistencyLoadErrorCode
{
    Unknown,
    Successful,

    JSON_NullOrEmptyString,
    JSON_ErrorDuringLoad,

    PersistedData_NullInput,
    PersistedData_WrongDataType,

    BuildingElementList_NullInput,
    BuildingElementList_MissingElement,

    BuildingElementConnection_MissingBuilding,
    BuildingElementConnection_MissingBuildingElement,

    Building_UnknownInstantiateError,
    Building_AlreadyUsedPosition,

    BuildingElement_UnknownInstantiateError,
    BuildingElement_UnknownComponentError,
    BuildingElement_NullPrefab,
    BuildingElement_AddToBuildingFailed,

    Update_WrongBasicObjectValues_BuildingParent,
    Update_WrongBasicObjectValues_Building,
    Update_WrongBasicObjectValues_BuildingElement,
}
```